ASSIGNMENT-10 : DESCRIBING VAPT AND VAPT REPORT WRITING

GUIDED BY : VIPUL MISTRI

Q.1 Write a brief explanation (in your own words) of what VAPT (Vulnerability Assessment and Penetration Testing) is, and list two differences between Vulnerability Assessment and Penetration Testing.

ANS : VAPT (Vulnerability Assessment and Penetration Testing) is a comprehensive cybersecurity practice that combines two complementary testing methodologies to secure networks, applications, and systems.

- **Vulnerability Assessment (VA)** is a broad, automated scan designed to discover and catalog known security weaknesses, missing patches, and misconfigurations across an entire IT infrastructure.

- **Penetration Testing (PT)** is a targeted, manual simulation of a real-world cyberattack where security professionals actively attempt to exploit those discovered flaws to determine how deep an adversary could infiltrate and what business impact it would cause.

Key Differences Between Vulnerability Assessment and Penetration Testing

Feature	**Vulnerability Assessment (VA)**	**Penetration Testing (PT)**
Approach & Execution	Primarily automated using scanners (e.g., Nessus, OpenVAS) to sweep targets quickly.	Highly manual, leveraging human creativity, custom scripts, and multi-step exploit chains.
Primary Objective	To identify, quantify, and list *all* potential security flaws and compliance gaps across the environment.	To demonstrate real-world risk by proving whether vulnerabilities can be successfully weaponized to compromise data.

ASSIGNMENT-10 : DESCRIBING VAPT AND VAPT REPORT WRITING

GUIDED BY : VIPUL MISTRI

Q.2 Imagine you are performing a VAPT for a food delivery app like Zomato. Identify and describe three common vulnerabilities you would check for in such an app.

ANS : When conducting a Vulnerability Assessment and Penetration Testing (VAPT) engagement for a high-traffic food delivery platform like Zomato, security testers focus heavily on API endpoints, business logic, user authorization controls, and transaction workflows.

Here are three common and critical vulnerabilities checked during a food delivery app assessment:

1. Insecure Direct Object References (IDOR / BOLA)

- **What it is:** IDOR occurs when an application exposes a reference to an internal implementation object (such as an order_id, user_id, or address_id) without sufficient authorization checks.

- **How it manifests in food delivery apps:** A user intercepts their own API request when viewing an order receipt (e.g., GET /api/v1/orders?order_id=45892) and simply changes the parameter to 45893. If the backend fails to verify whether the authenticated user owns that order, they gain unauthorized access to another customer's delivery address, phone number, order history, and partial payment details.

- **Testing Methodology:** Testers use proxy tools like Burp Suite to manipulate object identifiers across different user sessions to verify if horizontal privilege escalation is possible.

2. Broken Authentication & Weak OTP Rate Limiting

- **What it is:** Flaws in how authentication tokens, password resets, or One-Time Passwords (OTPs) are generated, validated, or throttled.

- **How it manifests in food delivery apps:** Delivery apps rely heavily on mobile-number login via OTP. If the endpoint handling OTP verification lacks proper rate limiting, an attacker can automate a brute-force attack against the 4-digit or 6-digit code. Similarly, if OTPs are predictable or leaked in API response headers during testing, accounts can be easily hijacked.

- **Testing Methodology:** Testers use automated intruder modules to bombard login and OTP verification endpoints with multiple parallel requests while monitoring for lockout mechanisms or rate-limiting error codes (such as HTTP 429 Too Many Requests).

3. Business Logic Flaws & Cart/Price Tampering

- **What it is:** Flaws in the design and implementation of application workflows that allow users to manipulate app behavior in unintended ways to gain financial or operational advantages.

- **How it manifests in food delivery apps:** E-commerce and food apps often rely on client-side calculation or trust input parameters sent from the mobile app to the server. An attacker might intercept the checkout request to modify item quantities to negative numbers, alter discount coupon values, or change the final payable amount to zero before submitting the order to the payment gateway.

- **Testing Methodology:** Testers map out multi-step user workflows (cart → discount application → payment) and deliberately inject unexpected numerical inputs, negative values, or expired promotional tokens into JSON payloads to see how the backend validates transaction integrity.

Q.3 Create a sample VAPT report template in a Word or Google Docs file for a fictional e-wallet app (like Paytm). Your template should include sections for Executive Summary, Methodology, Vulnerabilities Found, Risk Ratings, and Recommendations.

ANS : Understood! I will keep everything right here in our chat and provide all templates and details directly as formatted text without creating any document files.

ASSIGNMENT-10 : DESCRIBING VAPT AND VAPT REPORT WRITING

GUIDED BY : VIPUL MISTRI

Here is the complete **VAPT Assessment Report Template for a Fictional E-Wallet App (PaySecure)** structured in clean markdown for easy reading and copying:

VAPT Assessment Report: PaySecure E-Wallet App

* **Target Organization:** PaySecure Technologies Pvt. Ltd.

* **Assessment Period:** October 25, 2026 – November 01, 2026

* **Assessment Type:** Mobile Application & Web API Vulnerability Assessment and Penetration Testing (VAPT)

* **Report Version:** 1.0 (Final)

1. Executive Summary

PaySecure Technologies engaged the Cyber Security Assessment Team to perform a comprehensive Vulnerability Assessment and Penetration Testing (VAPT) exercise on its flagship e-wallet platform, **PaySecure**. The scope included the Android/iOS mobile application interfaces, backend RESTful APIs, and administrative merchant portals.

The primary objective was to identify security vulnerabilities that could be exploited by malicious actors to compromise user financial data, manipulate transaction balances, bypass authentication, or disrupt wallet operations.

Key Assessment Findings

* **Critical Risk:** An Insecure Direct Object Reference (IDOR / BOLA) flaw allowed horizontal privilege escalation, enabling authenticated users to view arbitrary user transaction histories and personal identifiable information (PII).

* **High Risk:** Lack of strict backend rate-limiting on the mobile OTP verification endpoint enabled automated brute-force attacks on login sessions.

* **Medium Risk:** Intercepted checkout payloads permitted client-side parameter tampering to alter wallet top-up discount values.

Overall, while perimeter transport security (HTTPS/TLS) and static code hardening were effectively enforced, severe business-logic and authorization weaknesses in API endpoints require immediate remediation prior to production feature releases.

2. Assessment Methodology

The assessment followed a hybrid testing model combining automated vulnerability scanning with extensive manual penetration testing, aligned with industry standards including **OWASP Mobile Top 10**, **OWASP API Security Top 10**, and **PTES (Penetration Testing Execution Standard)**.

Testing Phases

1. **Reconnaissance & Scope Definition:** Mapping API endpoints, analyzing APK/IPA binaries, and traffic interception setup via Burp Suite.

2. **Automated Vulnerability Scanning:** Running automated static analysis (SAST) on application binaries and dynamic scanning (DAST) on web/API endpoints.

3. **Manual Exploitation & Logic Testing:** Deep-dive manual testing focusing on authorization controls, session management, transaction logic, and parameter tampering.

4. **Risk Rating & Reporting:** Categorizing findings based on CVSS v3.1 vector scores and detailing practical remediation guidance.

---

3. Risk Rating Framework

Vulnerabilities are rated based on the **Common Vulnerability Scoring System (CVSS v3.1)**:

| Severity Level | CVSS v3.1 Score Range | Impact Summary |

| --- | --- | --- |

| **Critical** | 9.0 – 10.0 | Immediate threat to core business logic, wallet funds, or complete infrastructure takeover. |

| **High** | 7.0 – 8.8 | Potential for significant data exposure, account takeover, or financial manipulation with minimal user interaction. |

| **Medium** | 4.0 – 6.9 | Flaws that could lead to partial data leakage or requires specific conditions/user interaction to exploit. |

| **Low / Info** | 0.1 – 3.9 | Minor information disclosure or configuration hardening suggestions without direct exploitation paths. |

---

4. Vulnerabilities Found

Finding VAPT-01: Insecure Direct Object Reference (IDOR) in Transaction History API

* **Severity:** **Critical** (CVSS 9.1)

* **Affected Endpoint:** `GET /api/v2/wallet/history?account_id={ID}`

* **Description:** The transaction history API endpoint failed to validate whether the `account_id` parameter matched the authenticated user's session token. By incrementing the `account_id` integer value in intercepted HTTP requests, an attacker could extract full wallet transaction logs, account balances, linked bank account numbers, and phone numbers of any PaySecure user.

* **Impact:** Mass exposure of sensitive customer financial records and PII.

Finding VAPT-02: Missing Rate Limiting on Mobile OTP Verification

* **Severity:** **High** (CVSS 8.2)

* **Affected Endpoint:** `POST /api/v1/auth/verify-otp`

* **Description:** The endpoint handling login OTP verification did not enforce IP-based or account-based rate limiting. An attacker automated a 4-digit numeric code brute-force attack (10,000 combinations) in under 45 seconds using custom scripts without triggering account lockout mechanisms.

* **Impact:** Full account takeover of target mobile numbers.

Finding VAPT-03: Cart Amount Parameter Tampering in Wallet Top-Up

* **Severity:** **Medium** (CVSS 6.5)

* **Affected Endpoint:** `POST /api/v1/checkout/add-money`

* **Description:** During wallet top-up transactions, the request payload contained client-side parameters for discount coupon amounts (`discount_amount=500`). Modifying this parameter manually allowed users to credit bonus funds beyond authorized promotional thresholds.

* **Impact:** Financial loss via unauthorized wallet balance inflation.

---

5. Summary of Recommendations

To secure the PaySecure platform against identified threats, the security team recommends implementing the following prioritized fixes:

1. **Enforce Object-Level Authorization Checks:** Implement strict server-side access control validation ensuring that the authenticated user identity (extracted from the validated JWT token) strictly owns the requested `account_id` resource before returning data.

2. **Implement API Rate Limiting & Throttling:** Deploy strict rate-limiting rules (e.g., maximum 3 failed OTP attempts per 5 minutes per IP/User ID) at both the API Gateway layer (e.g., NGINX / AWS WAF) and backend application logic.

3. **Server-Side Transaction Calculations:** Never trust client-submitted price, discount, or numerical values. All calculations for wallet top-ups, discounts, and payable amounts must occur server-side using validated product/promo IDs.

4. **Continuous Security Testing:** Integrate automated SAST/DAST checks into the CI/CD pipeline and conduct re-testing after remediations are implemented.

Q.4 Given the following scenario: During a VAPT of a movie ticket booking app, you discover that user passwords are stored in plain text in the database. Write a short VAPT report entry describing this vulnerability, its risk level, and a recommendation to fix it. Hint: Use clear, professional language and cover what the risk is and how to mitigate it.

ANS : Here is a professional VAPT report entry for the discovered plain-text password vulnerability:

Finding: Plain-Text Storage of User Passwords in Database

- **Severity: Critical** (CVSS 9.8)

- **Affected Component:** Backend Database / Authentication User Table (users.password_hash or equivalent column)

- **Description:** During the security assessment of the movie ticket booking application, database inspection revealed that user account passwords are being stored in clear, unencrypted plain text within the primary user authentication table. No hashing functions, cryptographic salts, or key derivation algorithms (such as bcrypt, Argon2, or PBKDF2) are implemented.

- **Impact:** If the underlying database is compromised through SQL injection, insider threat, unauthorized access, or server misconfiguration, an attacker instantly gains access to raw plaintext credentials for all registered users. Because individuals frequently reuse passwords across multiple online platforms, this flaw creates severe cascading security risks for customers and reputational liability for the organization.

- **Recommendation:**

1. **Immediate Hashing Implementation:** Modify the backend authentication logic to remove plain-text storage immediately. Implement a strong, salted cryptographic hashing algorithm designed for passwords, such as **Argon2id** or **bcrypt** (with an appropriate work factor/cost parameter).

2. **Credential Reset:** Force a mandatory password reset for all existing user accounts upon deployment of the fix, as prior plaintext values should be considered compromised.

3. **Data Sanitization:** Ensure database backup logs, staging environments, and logs are purged of any historical plaintext credential records.